

MiniDuckSimulator

Primera Etapa

- Crear la clase Duck con metodos abstractos.
- Crear las siguientes clases implementando en cada caso los metodos abstractos de Duck.
 - MallardDuck
 - RedHeadDuck
 - RubberDuck
- En Program.cs, en Main, crear las clases y llamar a los metodos.

Hasta aca, todo es como lo conocemos.

El problema e que por cada nueva clase de pato hay que implementar todos los metodos abstractos de Duck y ademas, si aparece una necesidad de cambio a cualquier clase hay que modificar el codigo y la idea es siempre:

"Identificar lo que varía y separarlo de lo que permanece igual"

Esto es un principio de diseño.

Segunda etapa

Agregamos el metodo fly() en la clase Duck implementado para que todos los patos puedan volar.

```
{  
    Console.WriteLine("I am a duck and I can fly! Suck it up!!!!");  
}
```

Vemos en Main que ahora los patos vuelan.

Y esto trae el problema de los patos de goma que no deberían volar. Inacceptable!!!

Entonces, en el caso de los patos de goma hay que sobre escribir (override) el método fly y adaptarlo.

Si hay que hacer esto en muchos casos, se viola el principio de diseño mencionado antes.

Tercera Etapa

El problema del metodo fly en la clase base es que se está agregando un comportamiento (behavior) a todos los patos que hereden de la clase base.

Esto es un problema...

Una solución, es como vimos en la Segunda Etapa, crear el método virtual (diferente de abstract), implementado pero, puede ser sobre escrito en cada caso como vimos en el pato de goma que implementa el método fly así:

```
public override void fly()
{
    Console.WriteLine("As a rubber duck, I can't fly unless you throw me away...");
}
```